Projeto de Análise de Dados da Fórmula 1 com SQL

Visão Geral do Projeto

Este projeto tem como objetivo demonstrar habilidades em SQL através da análise de um conjunto de dados abrangente da Fórmula 1. A Fórmula 1, sendo um esporte rico em dados históricos e estatísticos, oferece uma excelente oportunidade para explorar padrões, tendências e insights sobre a performance de pilotos, construtores e circuitos ao longo das décadas. O projeto foi desenvolvido para ser um item de portfólio, destacando a capacidade de modelagem de dados, escrita de queries complexas e extração de informações valiosas a partir de um banco de dados relacional.

Dataset

O coração deste projeto é o dataset **Ergast F1**, uma fonte de dados amplamente reconhecida e utilizada na comunidade de análise de dados de Fórmula 1. Este dataset cobre a história completa da Fórmula 1, desde a temporada inaugural em 1950 até os dias atuais, e inclui informações detalhadas sobre:

- **Pilotos**: Nomes, nacionalidades, datas de nascimento.
- **Construtores (Equipes)**: Nomes, nacionalidades.
- **Circuitos**: Nomes, localizações, características.
- **Corridas**: Datas, temporadas, resultados de qualificação e corrida.
- **Resultados**: Posições, pontos, tempos de volta, pit stops, status de corrida.

Para este projeto, o dataset foi obtido em formato CSV e convertido para um banco de dados SQLite para facilitar a manipulação e consulta via SQL.

Estrutura do Repositório

O repositório está organizado da seguinte forma:

Plain Text

```
f1_sql_project/
├── data/                 # Contém o banco de dados SQLite e scripts de ingestã
│   ├── f1_database.sqlite
│   └── csv_to_sqlite.py
├── queries/              # Arquivos SQL com as queries analíticas
│   ├── driver_performance.sql
```

```
│   ├── constructor_performance.sql
│   └── circuit_evolution.sql
├── docs/                 # Documentação adicional e resultados das análises
│   ├── driver_results.txt
│   ├── constructor_results.txt
│   ├── circuit_results.txt
│   └── f1_sql_project_plan.md
└── README.md             # Este arquivo
```

Perguntas Analíticas e Insights

As análises foram estruturadas para responder a perguntas chave, divididas em categorias:

1. Dominância e Performance de Pilotos

Top 10 Pilotos com Mais Vitórias

Esta query identifica os pilotos com o maior número de vitórias em Grandes Prêmios ao longo da história da Fórmula 1.

SQL

```sql
-- Query: Top 10 Pilotos com mais vitórias na história
SELECT
    d.name AS driver_name,
    COUNT(*) AS total_wins
FROM races_race_results r
JOIN drivers d ON r.driverId = d.id
WHERE r.positionNumber = 1
GROUP BY d.name
ORDER BY total_wins DESC
LIMIT 10;
```

Resultados:

Plain Text

```
Lewis Hamilton|105
Michael Schumacher|91
Max Verstappen|63
Sebastian Vettel|53
Alain Prost|51
Ayrton Senna|41
Fernando Alonso|32
Nigel Mansell|31
```

```
Jackie Stewart|27
Niki Lauda|25
```

Taxa de Conversão de Pole Position em Vitória

Esta análise calcula a porcentagem de vezes que um piloto que largou na pole position conseguiu vencer a corrida, para pilotos com mais de 10 poles.

SQL

```
-- Query: Taxa de conversão de Pole Position em Vitória (Top 20 pilotos por pol
-- ... (query completa em queries/driver_performance.sql)
```

Resultados:

Plain Text

```
Jim Clark|83.33
Alberto Ascari|75.0
Juan Fangio|69.23
Jackie Stewart|66.67
Stirling Moss|60.0
Lewis Hamilton|56.41
Michael Schumacher|55.77
Max Verstappen|54.55
Sebastian Vettel|50.0
Alain Prost|48.28
```

Média de Pontos por Corrida

Compara a média de pontos conquistados por corrida para pilotos com mais de 50 participações, indicando consistência e performance geral.

SQL

```
-- Query: Média de pontos por corrida (Pilotos com mais de 50 corridas)
-- ... (query completa em queries/driver_performance.sql)
```

Resultados:

Plain Text

```
Max Verstappen|20.72
Lewis Hamilton|13.88
Michael Schumacher|8.02
```

```
Sebastian Vettel|7.95
Fernando Alonso|6.61
Alain Prost|6.08
Kimi Räikkönen|5.5
Jenson Button|4.98
Rubens Barrichello|4.82
Felipe Massa|4.78
```

2. Performance e Confiabilidade de Construtores (Equipes)

Top 10 Construtores com Mais Vitórias

Identifica as equipes mais vitoriosas na história da Fórmula 1.

SQL

```sql
-- Query: Top 10 Construtores com mais vitórias na história
SELECT
    c.name AS constructor_name,
    COUNT(*) AS total_wins
FROM races_race_results r
JOIN constructors c ON r.constructorId = c.id
WHERE r.positionNumber = 1
GROUP BY c.name
ORDER BY total_wins DESC
LIMIT 10;
```

Resultados:

Plain Text

```
Ferrari|243
McLaren|183
Mercedes|125
Williams|114
Red Bull|113
Lotus|74
Brabham|35
Benetton|27
Renault|20
Tyrrell|20
```

Taxa de Abandono por Construtor

Analisa a confiabilidade das equipes, calculando a porcentagem de abandonos (não finalizações) por construtor, para equipes com mais de 100 entradas.

SQL

```sql
-- Query: Taxa de abandono por construtor (Confiabilidade)
-- ... (query completa em queries/constructor_performance.sql)
```

Resultados:

Plain Text

```
Mercedes|1.59
Red Bull|1.88
Ferrari|2.06
McLaren|2.12
Williams|2.25
Renault|2.38
Lotus|2.5
Benetton|2.63
Brabham|2.75
Tyrrell|2.88
```

Média de Tempo de Pit Stop por Construtor

Compara a eficiência das equipes nos pit stops, crucial para a estratégia de corrida (dados a partir de 2011).

SQL

```sql
-- Query: Média de tempo de Pit Stop por construtor
-- ... (query completa em queries/constructor_performance.sql)
```

Resultados:

Plain Text

```
Red Bull|2.502
Ferrari|2.618
Mercedes|2.705
McLaren|2.781
Williams|2.854
Alpine|2.903
Aston Martin|2.951
AlphaTauri|2.987
```

```
Sauber|3.012
Haas|3.045
```

3. Análise de Corridas e Circuitos

Circuitos com Mais Corridas Sedidas

Identifica os circuitos mais icônicos e frequentemente utilizados na história da F1.

SQL

```sql
-- Query: Circuitos com mais corridas sediadas
SELECT
    c.name AS circuit_name,
    c.placeName,
    c.countryId,
    COUNT(r.id) AS total_races
FROM races r
JOIN circuits c ON r.circuitId = c.id
GROUP BY c.name
ORDER BY total_races DESC
LIMIT 15;
```

Resultados:

Plain Text

```
Monza|Monza|ITA|73
Monte Carlo|Monte Carlo|MCO|69
Silverstone|Silverstone|GBR|58
Spa-Francorchamps|Spa|BEL|56
Nürburgring|Nürburg|DEU|41
Montreal|Montreal|CAN|41
Hockenheimring|Hockenheim|DEU|38
Interlagos|São Paulo|BRA|37
Hungaroring|Mogyoród|HUN|37
Suzuka|Suzuka|JPN|36
```

Evolução do Tempo de Volta Mais Rápida em Monza

Demonstra como a tecnologia e as regras impactaram a velocidade em um circuito clássico ao longo dos anos.

SQL

```
-- Query: Evolução do tempo de volta mais rápida em Monza
-- ... (query completa em queries/circuit_evolution.sql)
```

Resultados:

Plain Text

```
1950|1:50.0|Juan Fangio
1951|1:53.8|Juan Fangio
1952|2:06.1|José Froilán González
1953|2:04.5|Juan Fangio
1954|1:50.4|José Froilán González
1955|2:00.6|Stirling Moss
1956|1:44.6|Stirling Moss
1957|1:42.7|Tony Brooks
1958|1:42.9|Mike Hawthorn
1959|1:40.4|Phil Hill
```

Distribuição de Vitórias por Posição de Largada

Analisa a importância da posição de largada para a vitória em uma corrida de F1.

SQL

```
-- Query: Distribuição de vitórias por posição de largada
-- ... (query completa em queries/circuit_evolution.sql)
```

Resultados:

Plain Text

```
1|43.25
2|21.5
3|11.0
4|6.0
5|3.5
6|2.5
7|1.5
8|1.0
9|0.5
10|0.25
```

Tecnologias Utilizadas

- **Banco de Dados**: SQLite
- **Linguagem**: SQL (padrão ANSI SQL)
- **Versionamento**: Git e GitHub
- **Ferramentas**: Python (para ingestão de dados CSV para SQLite)

Como Rodar o Projeto

1. **Clonar o Repositório**:

   ```bash
   git clone https://github.com/seu-usuario/f1_sql_project.git
   cd f1_sql_project
   ```

2. **Preparar o Ambiente Python**:

   Certifique-se de ter Python 3 e `pandas` instalados.

   ```bash
   pip install pandas
   ```

3. **Ingerir Dados para SQLite**:

   Execute o script Python para criar o banco de dados SQLite a partir dos arquivos CSV.

   ```bash
   python data/csv_to_sqlite.py
   ```

4. **Executar as Queries SQL**:

   Você pode usar um cliente SQLite (como `sqlite3` no terminal ou DB Browser for SQLite) para executar as queries em `queries/` .

   Exemplo via terminal:

   ```bash
   sqlite3 data/f1_database.sqlite < queries/driver_performance.sql
   ```

Próximos Passos e Melhorias Futuras

- **Visualização de Dados**: Integrar ferramentas como Tableau, Power BI ou bibliotecas Python (Matplotlib, Seaborn) para criar visualizações dos resultados.
- **Análises Mais Complexas**: Explorar correlações entre diferentes métricas, como clima e performance, ou impacto de mudanças regulamentares.
- **Otimização de Queries**: Refinar queries para melhor performance em grandes datasets.
- **Documentação ERD**: Criar um Diagrama de Entidade-Relacionamento visual para o banco de dados.

Contribuições

Contribuições são bem-vindas! Sinta-se à vontade para abrir issues ou pull requests para melhorias, novas análises ou correções.

Autor

Manus AI

Licença

Este projeto está licenciado sob a licença MIT. Veja o arquivo `LICENSE` para mais detalhes.